

Lecture 9 - Conjugate Priors & The Naive Bayes Classifier

Code Implementation

Let's illustrate this example with code:

We saw the example where our input dataset $\{x_i\}_{i=1}^N$ is binary, where $x_i \in \{0,1\}$, for all i .

For each data sample x_i , we modeled its data likelihood as the Bernoulli distribution with parameter μ , i.e.

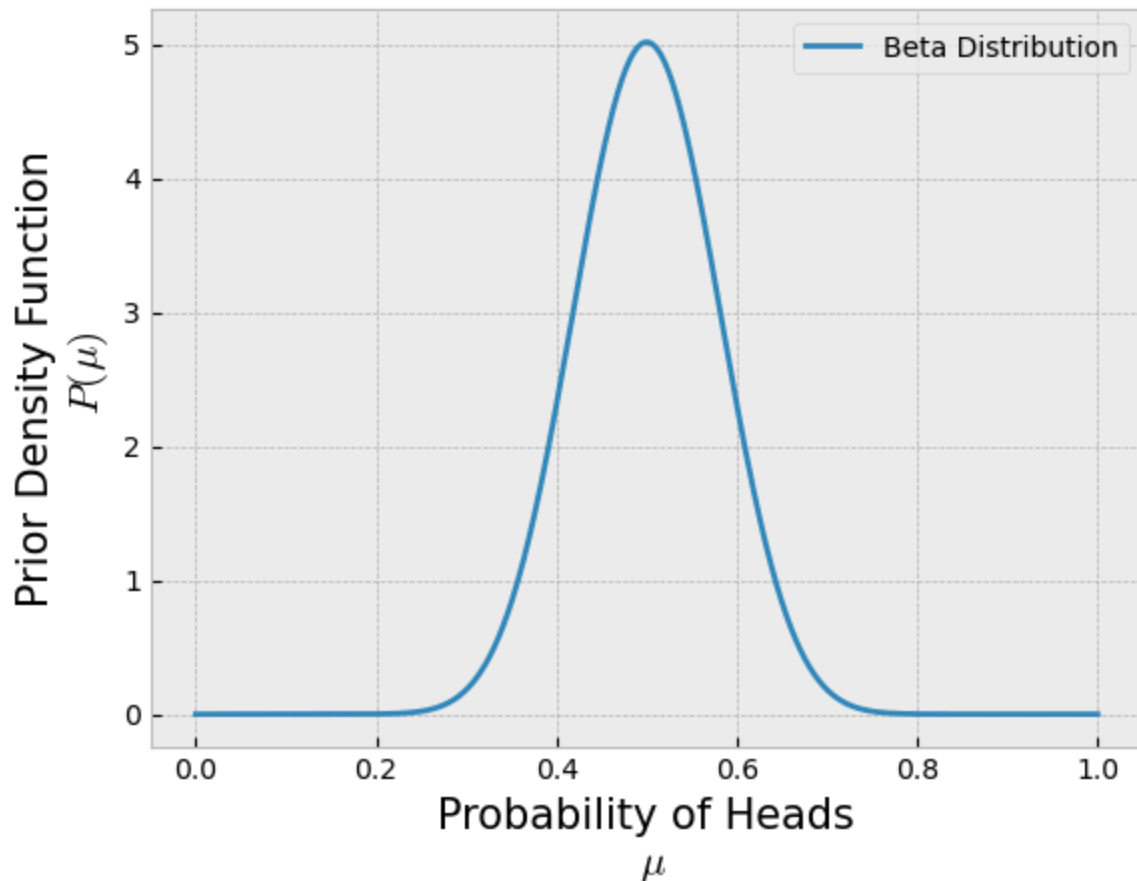
$$x_i \sim \text{Bernoulli}(\mu)$$

Furthermore, we assume that each sample is independent of each other. Hence our dataset is a collection of i.i.d. samples.

```
In [1]: import scipy.stats as stats
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')
import warnings
warnings.filterwarnings("ignore")
```

```
In [4]: ## Prior Probability
a = 20
b = 20
Beta = stats.beta(a,b)
x = np.linspace(0,1,1000)

plt.plot(x, Beta.pdf(x), label='Beta Distribution')
plt.legend(loc='best')
plt.xlabel('Probability of Heads\n  $\mu$ ', fontsize=15)
plt.ylabel('Prior Density Function\n  $P(\mu)$ ', fontsize=15);
```



```
In [5]: import scipy.stats as stats

trueMU = 0.5 # 0.5 for a fair coin

Nflips = 10

Outcomes = []
for i in range(Nflips):
    Outcomes += [stats.bernoulli(trueMU).rvs(1)[0]]
    print(Outcomes)
    print('MLE (Frequentist, data-driven): Probability of Heads = ', np.sum(
    print('MAP (Bayesian, uses prior): Probability of Heads = ', (np.sum(Out
    input('Press enter to flip the coin again...\n')

[1]
MLE (Frequentist, data-driven): Probability of Heads = 1.0
MAP (Bayesian, uses prior): Probability of Heads = 0.5128205128205128
[1, 1]
MLE (Frequentist, data-driven): Probability of Heads = 1.0
MAP (Bayesian, uses prior): Probability of Heads = 0.525
[1, 1, 0]
MLE (Frequentist, data-driven): Probability of Heads = 0.6666666666666666
MAP (Bayesian, uses prior): Probability of Heads = 0.5121951219512195
[1, 1, 0, 1]
MLE (Frequentist, data-driven): Probability of Heads = 0.75
MAP (Bayesian, uses prior): Probability of Heads = 0.5238095238095238
```

```

[1, 1, 0, 1, 0]
MLE (Frequentist, data-driven): Probability of Heads = 0.6
MAP (Bayesian, uses prior): Probability of Heads = 0.5116279069767442
[1, 1, 0, 1, 0, 0]
MLE (Frequentist, data-driven): Probability of Heads = 0.5
MAP (Bayesian, uses prior): Probability of Heads = 0.5
[1, 1, 0, 1, 0, 0, 1]
MLE (Frequentist, data-driven): Probability of Heads = 0.5714285714285714
MAP (Bayesian, uses prior): Probability of Heads = 0.5111111111111111
[1, 1, 0, 1, 0, 0, 1, 1]
MLE (Frequentist, data-driven): Probability of Heads = 0.625
MAP (Bayesian, uses prior): Probability of Heads = 0.5217391304347826
[1, 1, 0, 1, 0, 0, 1, 1, 0]
MLE (Frequentist, data-driven): Probability of Heads = 0.5555555555555556
MAP (Bayesian, uses prior): Probability of Heads = 0.5106382978723404
[1, 1, 0, 1, 0, 0, 1, 1, 0, 0]
MLE (Frequentist, data-driven): Probability of Heads = 0.5
MAP (Bayesian, uses prior): Probability of Heads = 0.5

```

```

In [7]: # True value of the unknown parameter
trueMU = 0.5 # 0.5 for a fair coin

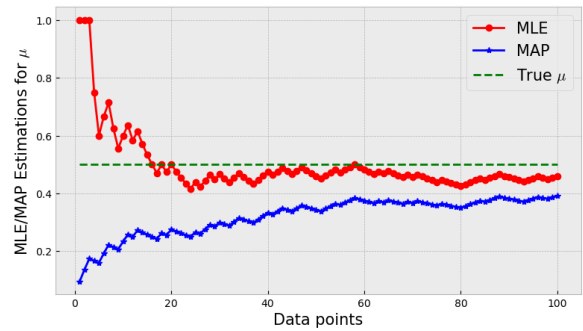
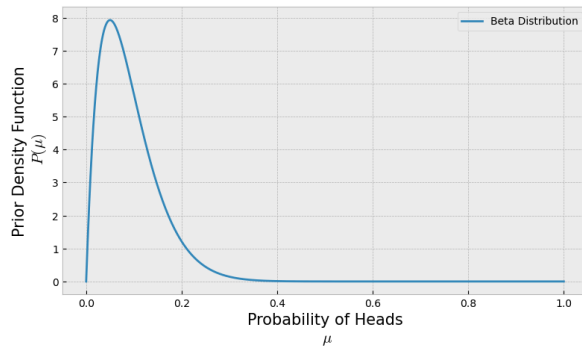
# Prior Initial Parameters
a=2 # alpha
b=20 # beta
# Plotting prior
plt.figure(figsize=(20,5))
plt.subplot(1,2,1)
Beta = stats.beta(a,b)
x = np.linspace(0,1,1000)
plt.plot(x, Beta.pdf(x), label='Beta Distribution')
plt.legend(loc='best')
plt.xlabel('Probability of Heads\n  $\mu$ ', fontsize=15)
plt.ylabel('Prior Density Function\n  $P(\mu)$ ', fontsize=15);

# Sampling Training Data
Nflips = 100
Outcomes = stats.bernoulli(trueMU).rvs(Nflips)

# Computing MLE and MAP estimates as data is being collected
mu_MLE = []
mu_MAP = []
for i in range(1,Nflips+1):
    mu_MLE += [np.sum(Outcomes[:i])/len(Outcomes[:i])]
    mu_MAP += [(np.sum(Outcomes[:i])+a-1)/(len(Outcomes[:i])+a+b-2)]

# Plotting estimates
plt.subplot(1,2,2)
plt.plot(range(1,Nflips+1), mu_MLE, '-or', label='MLE')
plt.plot(range(1,Nflips+1), mu_MAP, '-*b', label='MAP')
plt.plot(range(1,Nflips+1), [trueMU]*Nflips, '--g', label='True  $\mu$ ')
plt.xlabel('Data points', size=15)
plt.ylabel('MLE/MAP Estimations for  $\mu$ ', size=15)
plt.legend(fontsize=15);

```



Let's now see the effects on the posterior probability

```
In [16]: # True value of the unknown parameter
trueMU = 0.5 # 0.5 for a fair coin

# Prior Probability parameters
a = 2; a_init = a
b = 20; b_init = b

# Plotting data
x = np.linspace(-0.1,1.1,100)
xr = range(-1,3)

# Prior probability, Beta(a,b)
plt.plot(x, stats.beta(a,b).pdf(x))
plt.xlabel('$\mu$'); plt.ylabel('P($\mu$)')
plt.title('Initial Prior: alpha='+str(a)+' , beta='+str(b),size=15)
plt.show();

print('Alpha = ', a)
print('Beta = ', b)

Nsamples = [1,2,5,10,50,100,150,200,500]
Outcomes = stats.bernoulli(trueMU).rvs(size=Nsamples[-1])
for i in range(len(Nsamples)):
    Data = Outcomes[:Nsamples[i]]

    # Outcomes will have 1's or 0's (1 - Heads, 0 - Tails)
    estimate_mu = (np.sum(Data)+a-1)/(len(Data)+a+b-2)

    # Data Likelihood:
    fig=plt.figure(figsize=(15,5))
    fig.add_subplot(1,2,1)
    plt.stem(xr, stats.bernoulli(np.sum(Data)/len(Data)).pmf(xr))
    plt.xlabel('$x$'); plt.ylabel('P(X|$μ$)');
    plt.title('Data Likelihood, '+str(Nsamples[i])+' samples')

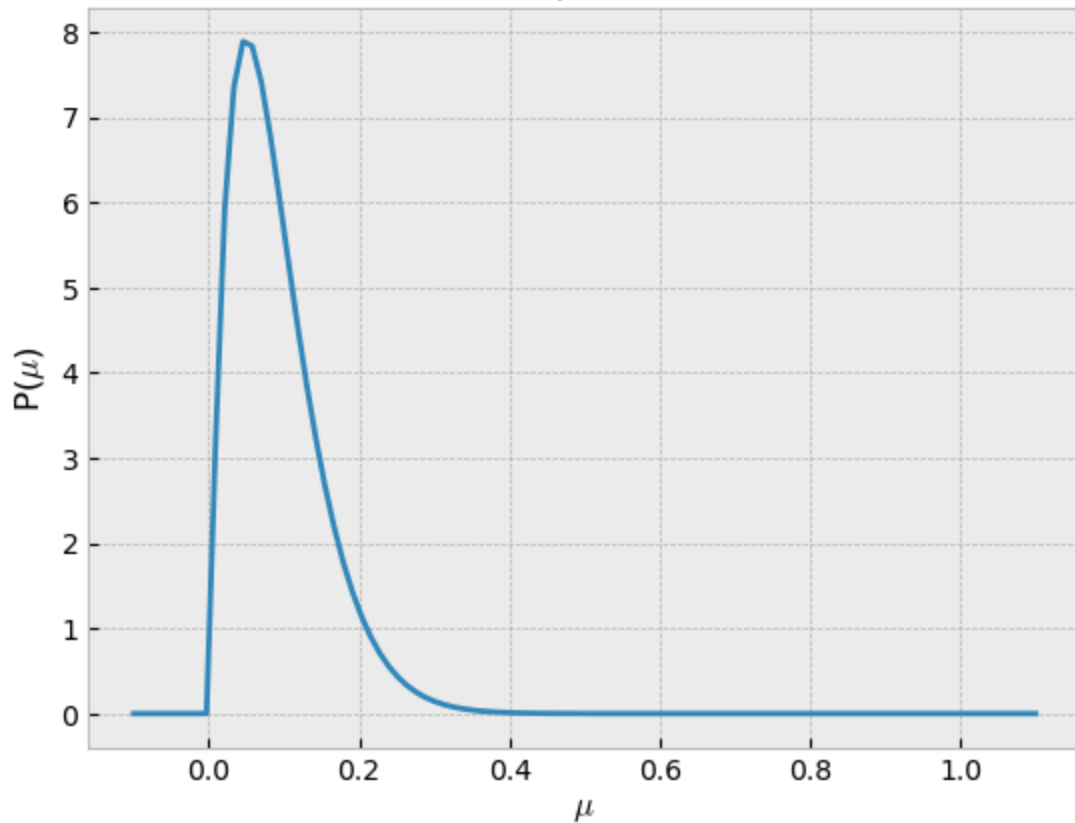
    # Posterior/Prior:
    fig.add_subplot(1,2,2)
    plt.plot(x, stats.beta(a,b).pdf(x))
    plt.xlabel('$μ$'); plt.ylabel('P($μ$|α, β)');
```

```
plt.title('Posterior/Prior: alpha='+str(a)+'', beta='+str(b))
plt.show()

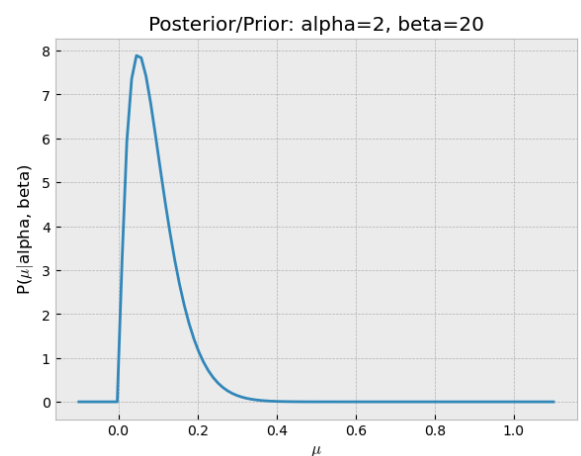
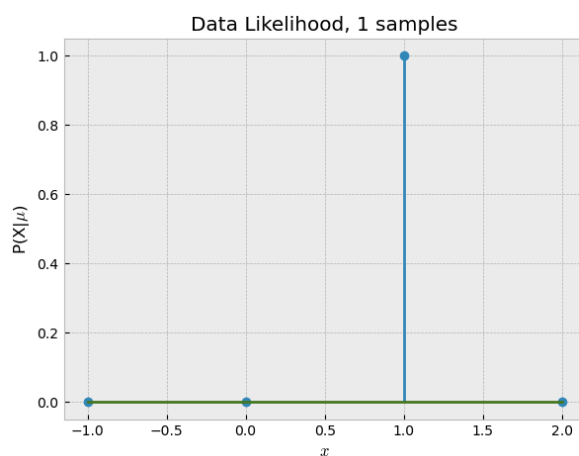
# Update Prior distribution
a += np.sum(Data)
b += len(Data)-np.sum(Data)

# Print estimate for mu
print('Number of samples: ', len(Data))
print('Data: ',Data)
print('MAP estimate mu = ', estimate_mu)
print('New alpha = ', a)
print('New beta = ', b)
```

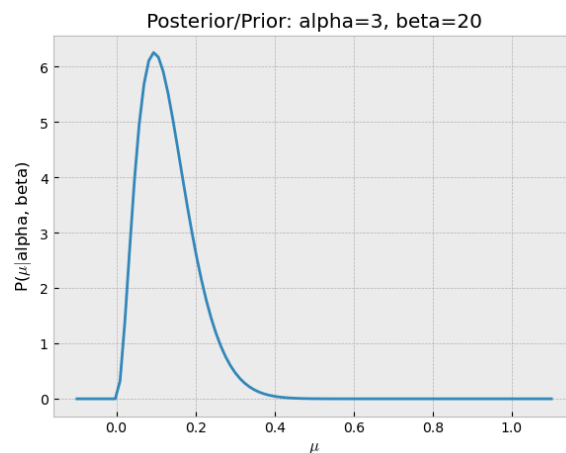
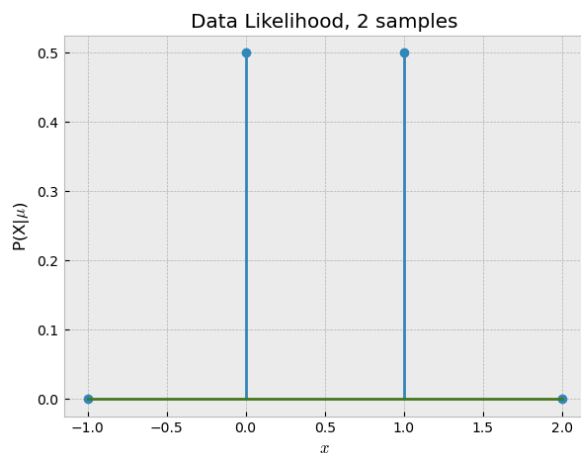
Initial Prior: alpha=2, beta=20



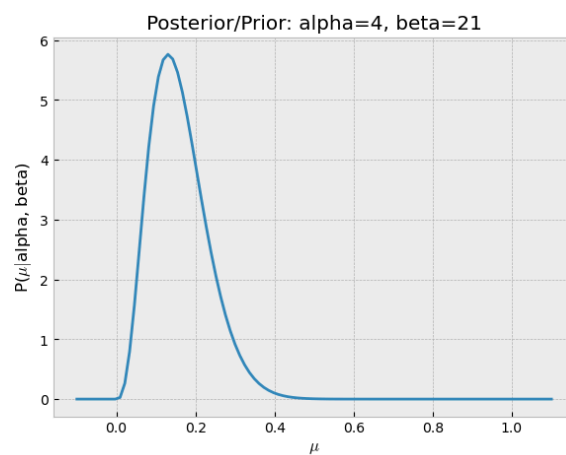
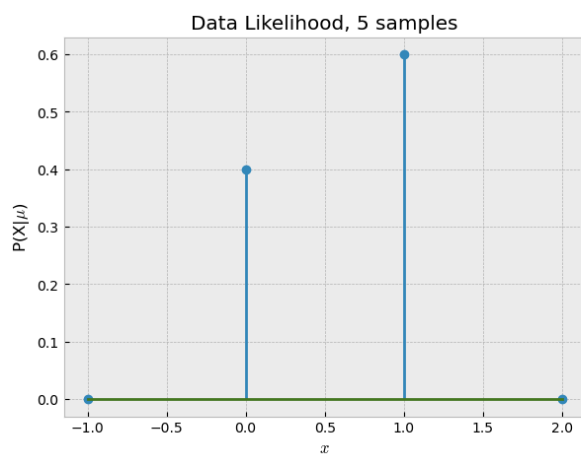
Alpha = 2
Beta = 20



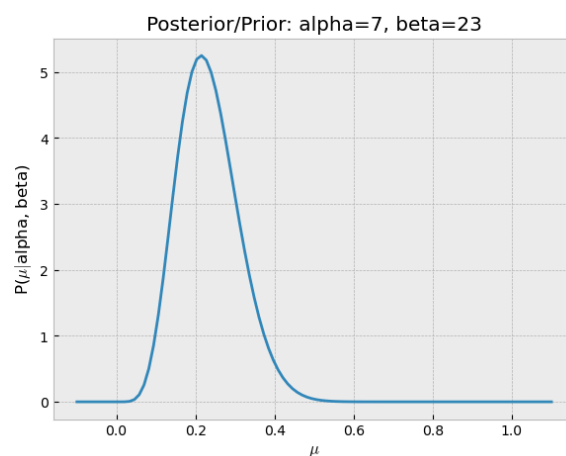
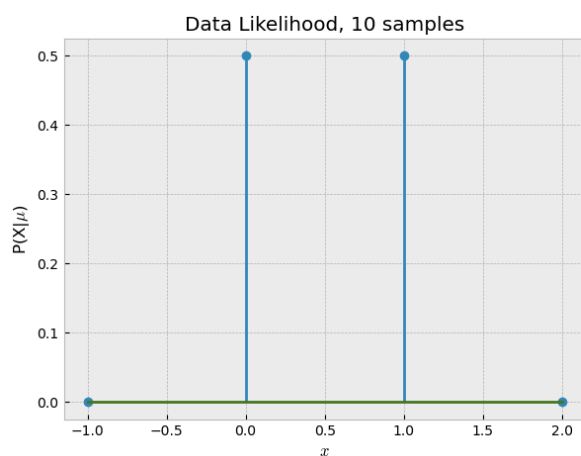
Number of samples: 1
Data: [1]
MAP estimate $\mu = 0.09523809523809523$
New alpha = 3
New beta = 20



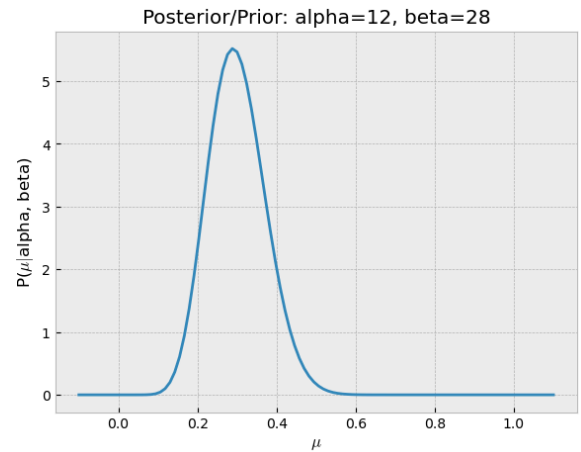
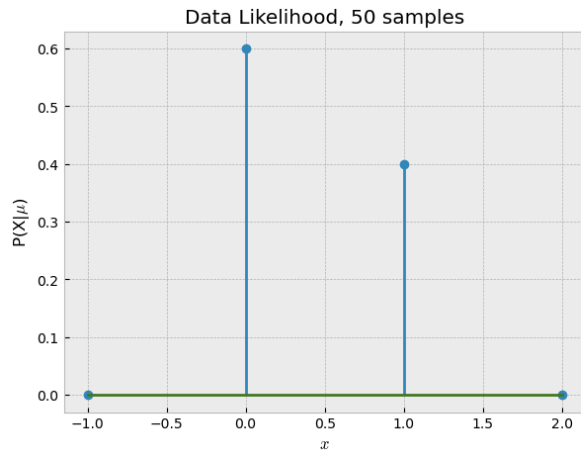
Number of samples: 2
Data: [1 0]
MAP estimate $\mu = 0.13043478260869565$
New alpha = 4
New beta = 21



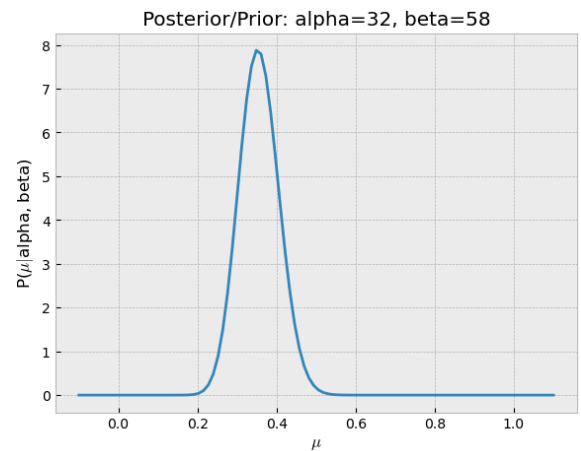
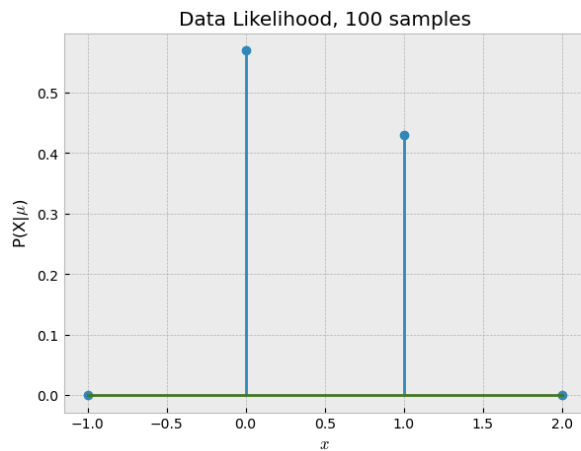
Number of samples: 5
Data: [1 0 1 0 1]
MAP estimate $\mu = 0.21428571428571427$
New alpha = 7
New beta = 23



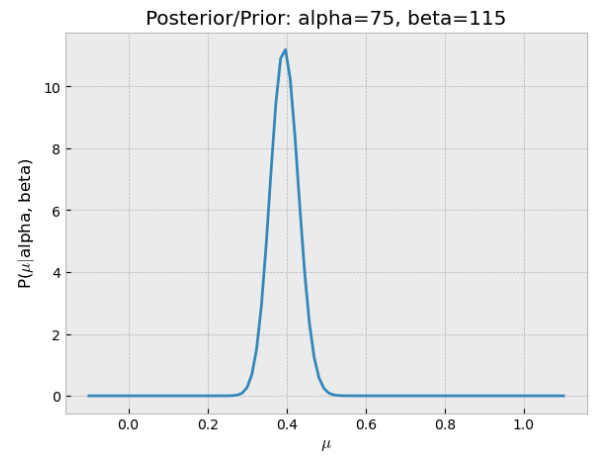
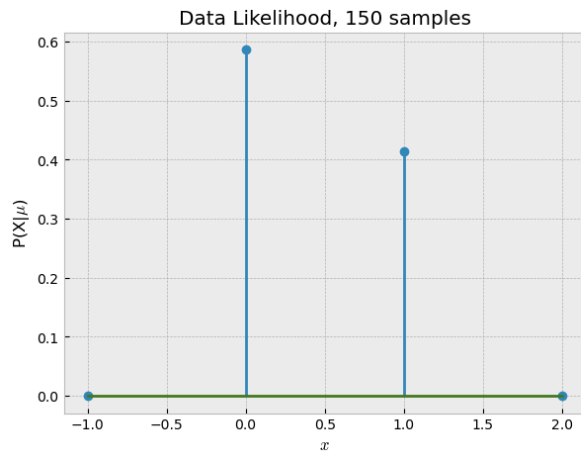
Number of samples: 10
 Data: [1 0 1 0 1 0 1 0 0 1]
 MAP estimate $\mu = 0.2894736842105263$
 New alpha = 12
 New beta = 28



Number of samples: 50
 Data: [1 0 1 0 1 0 1 0 0 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 0
 0 0 0
 1 0 1 0 0 0 0 1 0 0 0 1 1]
 MAP estimate $\mu = 0.3522727272727273$
 New alpha = 32
 New beta = 58



Number of samples: 100
 Data: [1 0 1 0 1 0 1 0 0 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 0
 0 0 0
 1 0 1 0 0 0 0 1 0 0 0 1 1 1 1 1 0 0 1 0 0 1 0 0 0 1 1 0 1 1 1 1 0 0 0 0 0
 1 1 0 0 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 0 1 1 0 1 1 1 1]
 MAP estimate $\mu = 0.39361702127659576$
 New alpha = 75
 New beta = 115



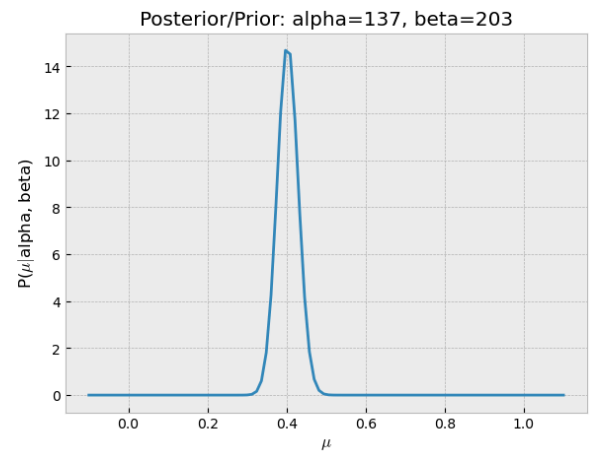
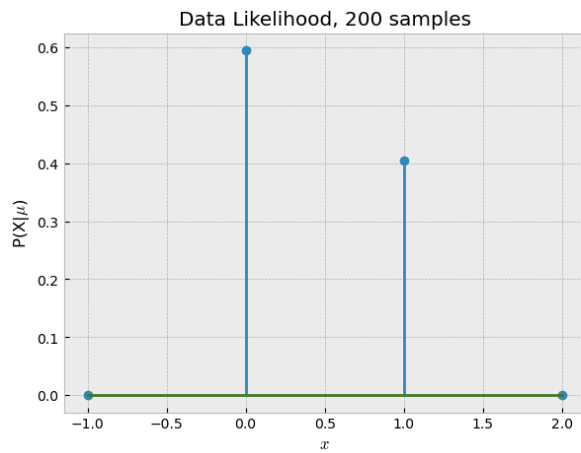
Number of samples: 150

Data: [1 0 1 0 1 0 1 0 0 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 0
0 0 0
1 0 1 0 0 0 0 1 0 0 0 1 1 1 1 1 0 0 1 0 0 1 0 0 0 1 1 0 1 1 1 1 0 0 0 0 0
1 1 0 0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 1 1 0 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0
0 1 0 1 0 0 1 0 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 1 0 1 1 1 0 0 1 1 0 0 0 0 1
0 0]

MAP estimate mu = 0.40236686390532544

New alpha = 137

New beta = 203



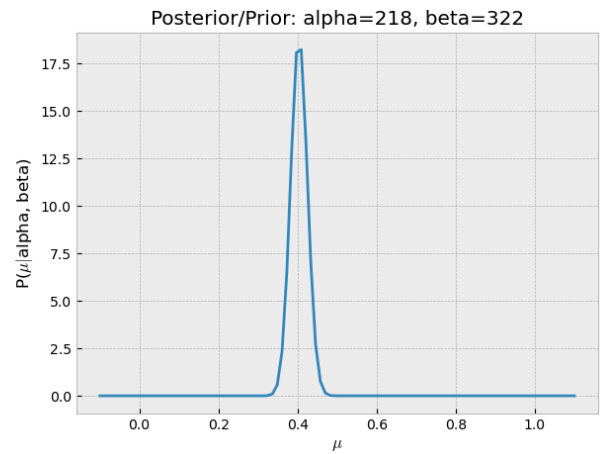
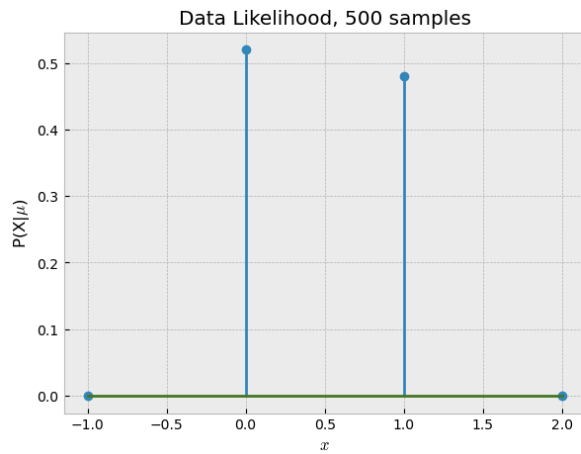
Number of samples: 200

Data: [1 0 1 0 1 0 1 0 0 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 0
0 0 0
1 0 1 0 0 0 0 1 0 0 0 1 1 1 1 1 0 0 1 0 0 1 0 0 0 1 1 0 1 1 1 1 1 0 0 0 0 0
1 1 0 0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 1 1 0 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0
0 1 0 1 0 0 1 0 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 1 0 1 1 1 0 0 1 1 0 0 0 0 1
0 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 1 0 0 0 1 1 1 0 1 0 1 1 0 0 1 1 0 0 1 0 0
0 0 0 1 0 0 1 0 0 1 0 1 1 1 0]

MAP estimate mu = 0.4033457249070632

New alpha = 218

New beta = 322



Number of samples: 500

Data: [1 0 1 0 1 0 1 0 0 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 0
0 0 0

```
1 0 1 0 0 0 0 1 0 0 0 1 1 1 1 1 0 0 1 0 0 1 0 0 0 1 1 0 1 1 1 1 0 0 0 0 0
1 1 0 0 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 0 1 1 0 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0
0 1 0 1 0 0 1 0 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 1 0 1 1 1 0 0 1 1 0 0 0 0 1
0 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 1 0 0 0 1 1 1 0 1 0 1 1 0 0 1 1 0 0 1 0 0
0 0 0 1 0 0 1 0 0 1 0 1 1 1 0 0 0 1 1 0 1 0 0 0 1 0 1 1 0 0 1 0 0 0 0 1 1
1 1 1 1 1 1 1 0 1 0 0 0 0 1 1 1 1 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0 1 1 1 1 0
1 1 1 0 0 1 1 1 1 0 1 1 1 1 0 1 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0 1 1 1 1 1
1 0 0 1 0 0 1 1 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 1 0 1 0 0 0 1 0
0 0 0 1 1 1 0 0 0 1 0 0 0 1 1 0 1 1 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 0 0 0 0
1 1 1 1 1 0 1 0 1 0 1 0 1 1 0 0 1 1 1 1 1 0 0 0 1 0 0 0 1 0 1 0 1 0 1 0
1 0 0 1 1 1 1 1 0 1 1 0 1 0 0 1 1 0 1 1 1 0 1 0 1 1 1 1 1 1 1 0 1 0 1 0 1
0 1 1 0 1 1 0 1 1 1 0 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 1 0 1 1
0 1 0 0 1 1 1 0 0 1 0 0 1 1 0 1 0 1 1]
```

MAP estimate $\mu = 0.44026974951830444$

New alpha = 458

New beta = 582

Other Conjugate Priors

There are many conjugate prior relationships, some examples include:

1. Bernoulli-Beta,
2. Gaussian-Gaussian,
3. Gaussian-Inverse Wishart,
4. Multinomial-Dirichlet,
5. and others.

The [table of conjugate distributions](#) is very useful for selecting the prior probability in order to have a conjugate prior relationship for cases when the data likelihood is discrete (such as Bernoulli) or continuous (such as Gaussian).

Could we update the prior as we receive data?

- What if we could fine-tune the prior probability's parameters (α and β , in this example) as we see more data?
 - Could we use the posterior probability to update the prior probability's parameters? That is, to select new values for α and β using a data informative prior?
 - What cases would make this possible?
-

Conjugate Priors

Two distributions have a **conjugate prior** relationship when the form of the posterior is the same as the form of the prior.

- Do the **Bernoulli-Beta** distributions have a conjugate prior relationship? Yes!

Note that the posterior probability was defined as:

$$\begin{aligned} P(\mu | \mathbf{x}) &\propto \mu^{\sum_{i=1}^N x_i + \alpha - 1} (1 - \mu)^{N - \sum_{i=1}^N x_i + \beta - 1} \end{aligned}$$

and, the prior probability (Beta distribution) is:

$$\begin{aligned} P(\mu | \alpha, \beta) &\propto \mu^{\alpha - 1} (1 - \mu)^{\beta - 1} \end{aligned}$$

(In both cases, the constant term was disregarded because it will not affect the solution for μ during optimization).

The **posterior** and the **prior** probability have the same shape, hence they have a **conjugate prior** relationship. Moreover, see that the parameters α and β are now mapped to:

$$\begin{aligned} \alpha^{(t+1)} &\leftarrow \alpha^{(t)} + \sum_{i=1}^N x_i \\ \beta^{(t+1)} &\leftarrow \beta^{(t)} + N - \sum_{i=1}^N x_i \end{aligned}$$

In an online model estimation scenario, where the posterior has the same form as the prior, we can use the posterior as our new prior. This new prior is now data informative and will update it's parameters based on (1) our initial choice, and (2) the data.

Pseudo-Code for Online Update of the Prior

1. Iteration $t=0$
2. Initialize the parameters of the prior probability, $\alpha^{(t)}$ and $\beta^{(t)}$
3. As data comes in:
 - A. Compute the posterior probability, $\mathcal{L}_{\text{MAP}}^{(t)} = P(\mu | \alpha^{(t)}, \beta^{(t)})$
 - B. Make an estimate for the parameter, $\mu_{\text{MAP}}^{(t)}$
 - C. Update parameters of prior probability:

$$\begin{aligned} \alpha^{(t+1)} &\leftarrow \alpha^{(t)} + \sum_{i=1}^N x_i \\ \beta^{(t+1)} &\leftarrow \beta^{(t)} + N - \sum_{i=1}^N x_i \end{aligned}$$
4. $t \leftarrow t + 1$

Let's see how we can implement this in code for this working example:

```
In [20]: # True value of the unknown parameter
trueMU = 0.5 # 0.5 for a fair coin

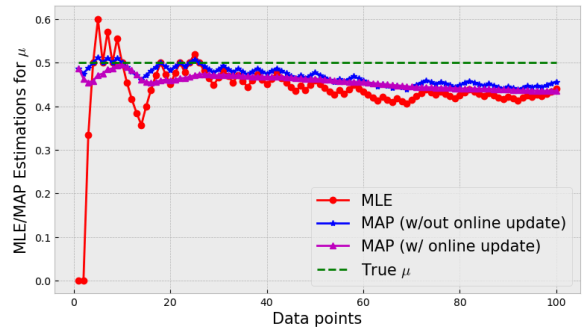
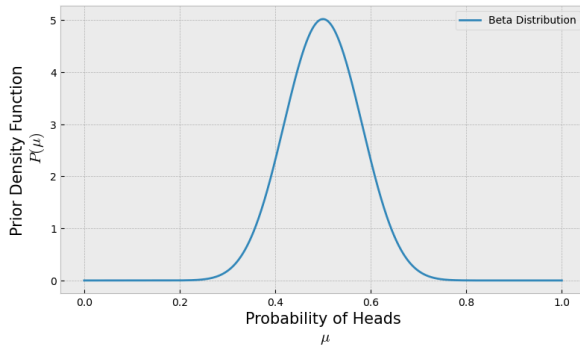
# Prior Probability parameters
a = 20; a_init = a
b = 20; b_init = b
# Plotting prior
plt.figure(figsize=(20,5))
plt.subplot(1,2,1)
Beta = stats.beta(a_init,b_init)
x = np.linspace(0,1,1000)
plt.plot(x, Beta.pdf(x), label='Beta Distribution')
plt.legend(loc='best')
plt.xlabel('Probability of Heads\n $μ$', fontsize=15)
plt.ylabel('Prior Density Function\n $P(μ)$', fontsize=15);

# Sampling Training Data
Nflips = 100
Outcomes = stats.bernoulli(trueMU).rvs(Nflips)

# Computing MLE and MAP estimates as data is being collected
mu_MLE = []
mu_MAP = []
mu_MAP_update = []
for i in range(1,Nflips+1):
    mu_MLE += [np.sum(Outcomes[:i])/len(Outcomes[:i])]
    mu_MAP += [(np.sum(Outcomes[:i])+a_init-1)/(len(Outcomes[:i])+a_init+b_init-2)]
    mu_MAP_update += [(np.sum(Outcomes[:i])+a-1)/(len(Outcomes[:i])+a+b-2)]
    a += np.sum(Outcomes[:i])
    b += len(Outcomes[:i]) - np.sum(Outcomes[:i])

# Plotting estimates
plt.subplot(1,2,2)
plt.plot(range(1,Nflips+1), mu_MLE, '-or', label='MLE')
plt.plot(range(1,Nflips+1), mu_MAP, '-*b', label='MAP (w/out online update)')
plt.plot(range(1,Nflips+1), mu_MAP_update, '^m', label='MAP (w/ online update)')
plt.plot(range(1,Nflips+1), [trueMU]*Nflips, '--g', label='True $μ$')
plt.xlabel('Data points',size=15)
```

```
plt.ylabel('MLE/MAP Estimations for  $\mu$ ',size=15)
plt.legend(fontsize=15);
```



Let's now see the effects on the posterior probability

```
In [ ]: # True value of the unknown parameter
trueMU = 0.5 # 0.5 for a fair coin

# Prior Probability parameters
a = 2; a_init = a
b = 20; b_init = b

# Plotting data
x = np.linspace(-0.1,1.1,100)
xr = range(-1,3)

# Prior probability, Beta(a,b)
plt.plot(x, stats.beta(a,b).pdf(x))
plt.xlabel('$\mu$'); plt.ylabel('P($\mu$)')
plt.title('Initial Prior: alpha='+str(a)+' , beta='+str(b),size=15)
plt.show();

print('Alpha = ', a)
print('Beta = ', b)

Nsamples = [1,2,5,10,50,100,150,200,500]
Outcomes = stats.bernoulli(trueMU).rvs(size=Nsamples[-1])
for i in range(len(Nsamples)):
    Data = Outcomes[:Nsamples[i]]

    # Outcomes will have 1's or 0's (1 - Heads, 0 - Tails)
    estimate_mu = (np.sum(Data)+a-1)/(len(Data)+a+b-2)

    # Data Likelihood:
    fig=plt.figure(figsize=(15,5))
    fig.add_subplot(1,2,1)
    plt.stem(xr, stats.bernoulli(np.sum(Data)/len(Data)).pmf(xr))
    plt.xlabel('$x$'); plt.ylabel('P(X|$mu$)');
    plt.title('Data Likelihood, '+str(Nsamples[i])+' samples')

    # Posterior/Prior:
    fig.add_subplot(1,2,2)
    plt.plot(x, stats.beta(a,b).pdf(x))
```

```
plt.xlabel('$\mu$'); plt.ylabel('P($\mu|\alpha, \beta$)');
plt.title('Posterior/Prior: alpha='+str(a)+' , beta='+str(b))
plt.show()

# Update Prior distribution
a += np.sum(Data)
b += len(Data)-np.sum(Data)

# Print estimate for mu
print('Number of samples: ', len(Data))
print('Data: ',Data)
print('MAP estimate mu = ', estimate_mu)
print('New alpha = ', a)
print('New beta = ', b)
```

Other Conjugate Priors

There are many conjugate prior relationships, some examples include:

1. Bernoulli-Beta,
2. Gaussian-Gaussian,
3. Gaussian-Inverse Wishart,
4. Multinomial-Dirichlet,
5. and others.

The [table of conjugate distributions](#) is very useful for selecting the prior probability in order to have a conjugate prior relationship for cases when the data likelihood is discrete (such as Bernoulli) or continuous (such as Gaussian).

Maximum Likelihood Estimation (MLE)

(Frequentist approach)

$$\arg_{\mathbf{w}} \max P(\mathbf{t}|\mathbf{w})$$

In **Maximum Likelihood Estimation** we *find the set of parameters* that **maximize** the data likelihood $P(\mathbf{t}|\mathbf{w})$. We find the *optimal* set of parameters under some assumed distribution such that the data is most likely.

- MLE focuses on maximizing the data likelihood, which *usually* provides a pretty good estimate
- A common trick to maximize the data likelihood is to maximize the log likelihood
- MLE is purely data driven

- MLE works best when we have lots and lots of data
- MLE will likely overfit when we have small amounts of data or, at least, becomes unreliable
- It estimates relative frequency for our model parameters. Therefore it needs incredibly large amounts of data (infinite!) to estimate the true likelihood parameters
 - This is a problem when we want to make inferences and/or predictions outside the range of what the training data has learned

Maximum A Posteriori (MAP)

(Bayesian approach)

$$\begin{aligned} & \arg_{\mathbf{w}} \max P(\mathbf{t}|\mathbf{w})P(\mathbf{w}) \quad \& \\ \propto & \arg_{\mathbf{w}} \max P(\mathbf{w}|\mathbf{t}) \end{aligned}$$

In **Maximum A Posteriori** we *find the set of parameters* that **maximize** the the posterior probability $P(\mathbf{w}|\mathbf{t})$. We find the *optimal* set of parameters under some assumed distribution such that the parameters are most likely to have been drawn off of.

- MAP focuses on maximizing the posterior probability - data likelihood with a prior
- A common trick to maximize the posterior probability is to maximize the log likelihood
- MAP is data driven
- MAP is mostly driven by the prior beliefs
- MAP works great with small amounts of data *if* our prior was chosen well
- We need to assume and select a distribution for our prior beliefs
 - A wrong choice of prior distribution can impact negatively our model estimation
- When we have lots and lots of data, the data likelihood will take over and the posterior will depend less and less on the prior

Review of Gaussian Distribution

(Read [section 2.3 "The Gaussian Distribution"](#) from the Bishop textbook)

Univariate Gaussian

The Gaussian distribution is a widely used probabilistic model for the probability density function (pdf) of continuous random variables.

The Gaussian distribution can model both univariate (1-D) or multivariate (multi-dimensional) samples.

In the **univariate** case, the pdf of a Gaussian distribution for a random variable $X \in \mathbb{R}$ can be written as

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

In this case, we say that X follows a Gaussian distribution with mean μ and variance σ^2 , or, $X \sim N(\mu, \sigma^2)$.

- We can define the **precision** parameter β as the inverse of the variance, that is, $\beta = \frac{1}{\sigma^2}$.
- A Gaussian distribution is called **Normal** when the mean is $\mu=0$ and variance is $\sigma^2=1$, $X \sim N(0,1)$.

```
In [22]: import scipy.stats as stats

G1=stats.norm(0,1) # G(mean=0, variance=1^2)

G2=stats.norm(10,np.sqrt(8)) #G(mean=10, variance=8)

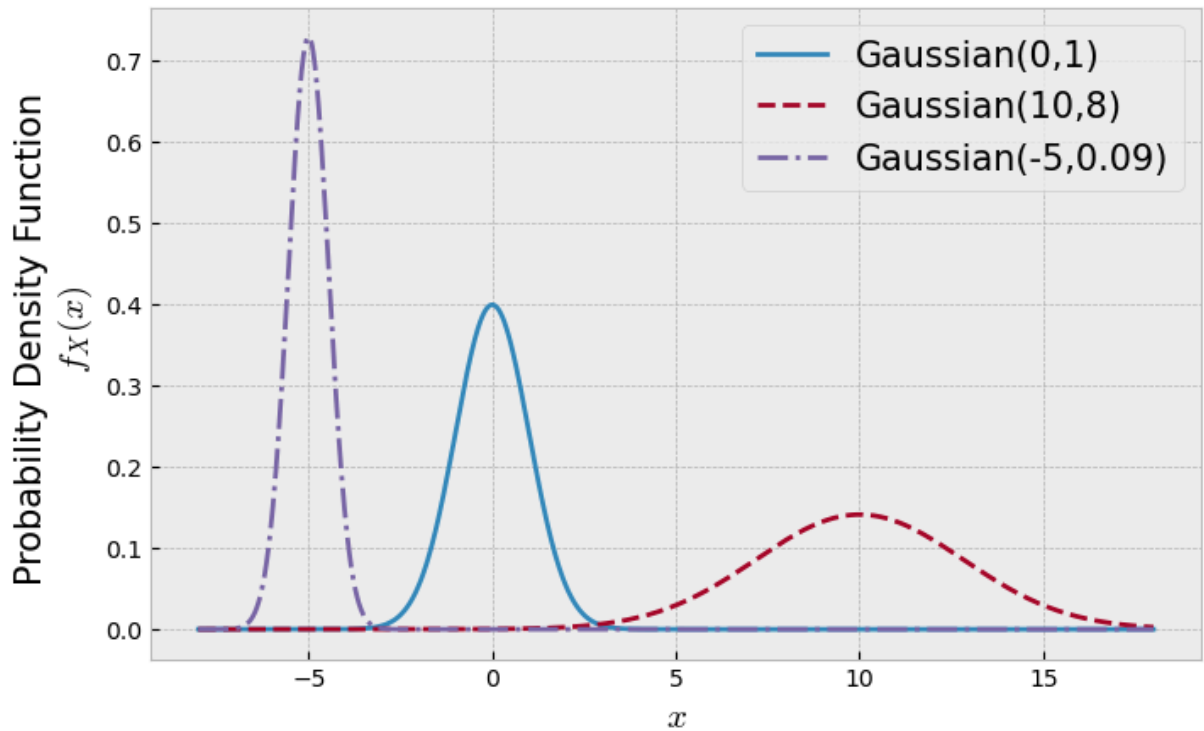
G3=stats.norm(-5,np.sqrt(0.3)) #G(mean=-5, variance=0.3^2)
```

```
In [24]: G1
```

```
Out[24]: <scipy.stats._distn_infrastructure.rv_continuous_frozen at 0x1569d9a30>
```

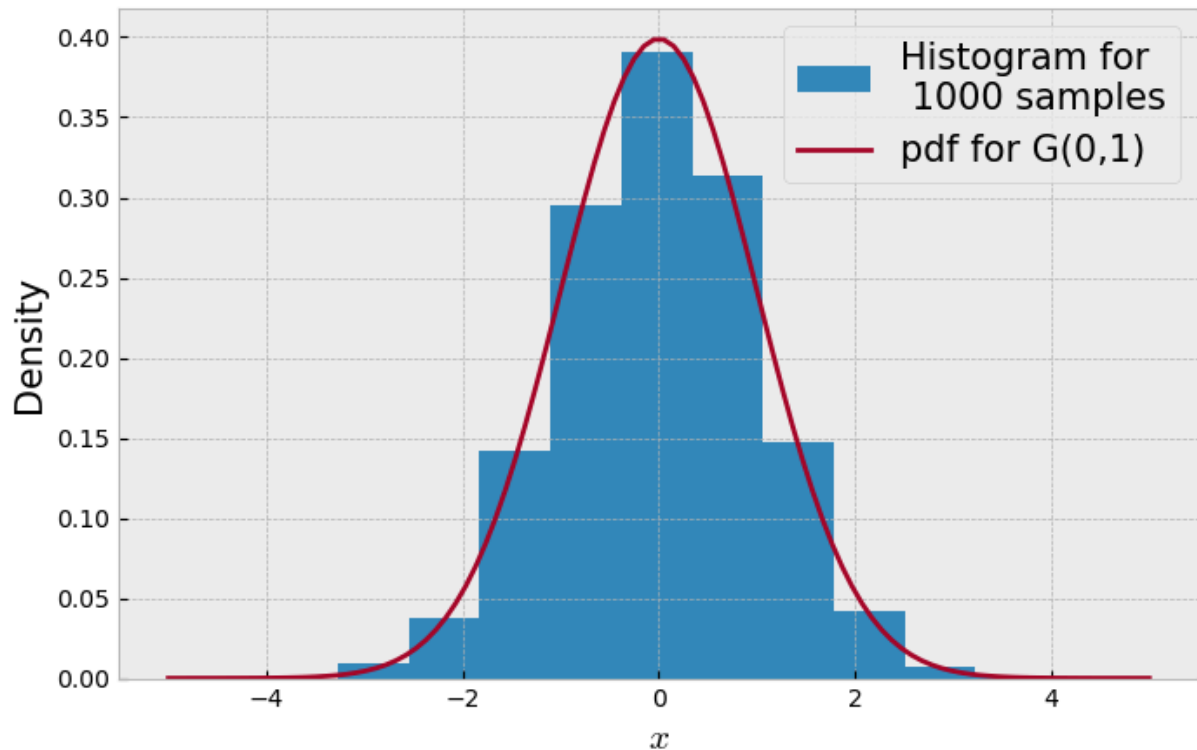
```
In [23]: plt.figure(figsize=(8,5))

x=np.linspace(-8,18,1000)
plt.plot(x,G1.pdf(x),label='Gaussian(0,1)')
plt.plot(x,G2.pdf(x),'--',label='Gaussian(10,8)')
plt.plot(x,G3.pdf(x),'-.',label='Gaussian(-5,0.09)')
plt.legend(fontsize=15)
plt.xlabel('$x$',size=15)
plt.ylabel('Probability Density Function \n $f_X(x)$',size=15);
```



```
In [26]: samples=G1.rvs(size=10_000)

plt.figure(figsize=(8,5))
x=np.linspace(-5,5,100)
plt.hist(samples,density=True, label='Histogram for\n 1000 samples')
plt.plot(x, G1.pdf(x), label='pdf for G(0,1)')
plt.legend(fontsize=15)
plt.xlabel('$x$',size=15)
plt.ylabel('Density',size=15);
```

Multivariate Gaussian

In the **multivariate** case, the pdf of a Gaussian distribution for a random variable $X \in \mathbb{R}^D$ can be written as

$$f_X(x) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)$$

In this case, we say that X follows a Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance Σ , or, $X \sim N(\boldsymbol{\mu}, \Sigma)$.

- $\boldsymbol{\mu}$ is a d -dimensional mean vector
- Σ is a $d \times d$ covariance matrix
- $|\Sigma|$ denotes the determinant of Σ
- The precision parameter in a d -dimensional space is equal to $\beta = \Sigma^{-1}$

Let $X = [X_1, X_2]$. The **covariance** Σ measures the amount of variance in each individual dimension, X_1 and X_2 , as well as the amount of covariance between the two. We can write the covariance as

$$\begin{aligned} \text{cov}(X_1, X_2) &= E[(X_1 - E[X_1])(X_2 - E[X_2])] \\ &= \begin{bmatrix} \text{var}(X_1) & \text{cov}(X_1, X_2) \\ \text{cov}(X_2, X_1) & \text{var}(X_2) \end{bmatrix} \\ &= \begin{bmatrix} \sigma^2_{X_1} & \text{cov}(X_1, X_2) \\ \text{cov}(X_1, X_2) & \sigma^2_{X_2} \end{bmatrix} \end{aligned}$$

- The **Pearson's correlation coefficient** between random variables X_1 and X_2 is defined as:

$$r = \frac{\text{cov}(X_1, X_2)}{\sqrt{\text{var}(X_1)}\sqrt{\text{var}(X_2)}} = \frac{\text{cov}(X_1, X_2)}{\sigma_{X_1} \sigma_{X_2}}$$

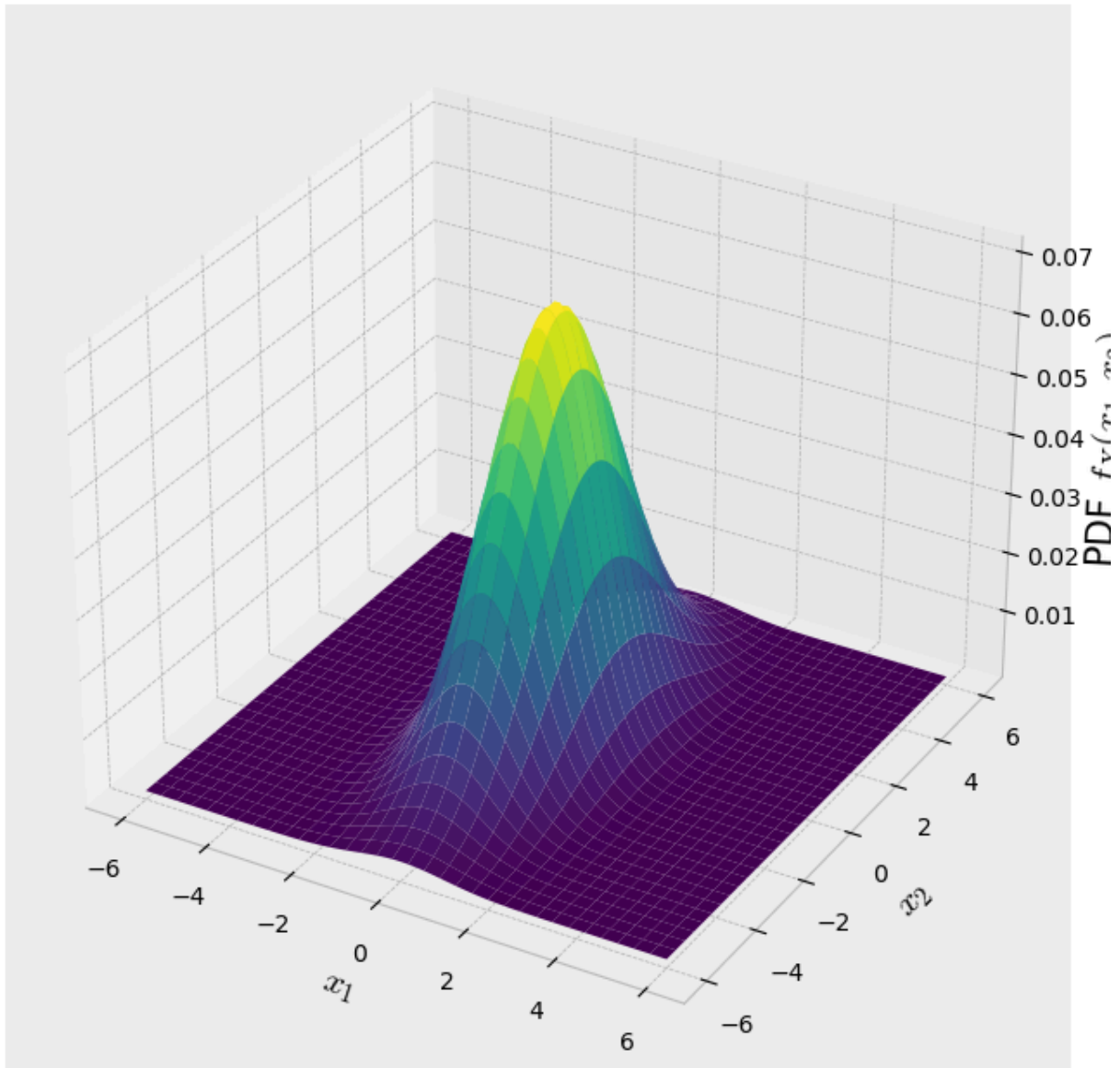
```
In [29]: mu = np.array([0,0]) # mean vector
cov = np.array([[1,0],[0,5]]) # covariance matrix
print(cov)
G = stats.multivariate_normal(mu, cov)
G
```

```
[[1 0]
 [0 5]]
```

```
Out[29]: <scipy.stats._multivariate.multivariate_normal_frozen at 0x16881b230>
```

```
In [30]: x, y = np.mgrid[-6:6:100j, -6:6:100j]
xy = np.column_stack([x.flat, y.flat])
z = stats.multivariate_normal.pdf(xy, mean=mu, cov=cov)
z = z.reshape(x.shape)

fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(x,y,z, rstride=3, cstride=3, linewidth=1, antialiased=True,
               cmap=plt.cm.viridis)
ax.set_xlabel('$x_1$', size=15)
ax.set_ylabel('$x_2$', size=15)
ax.set_zlabel('PDF $f_X(x_1, x_2)$', size=15);
```



Let's use the Mathematica's demonstration ["Joint Density of Bivariate Gaussian Random Variables"](#) to better understand the role of the covariance matrix.

Example: Multivariate Gaussian-Gaussian Conjugate Prior Relationship

For a D-dimensional Gaussian data likelihood with mean μ and covariance $\beta \mathbf{I}$ and a prior distribution with mean μ_0 and covariance Σ_0

$$\begin{aligned} P(\mathbf{t}|\mathbf{w}) &\sim \mathcal{N}(\mathbf{\mu}, \beta \mathbf{I}) \\ P(\mathbf{w}) &\sim \mathcal{N}(\mathbf{\mu}_0, \Sigma_0) \end{aligned}$$

The posterior distribution

$$\begin{aligned} P(\mathbf{w}|\mathbf{t}) &\sim \mathcal{N}(\mathbf{\mu}_N, \Sigma_N) \\ \mathbf{\mu}_N &= \Sigma_N^{-1} (\Sigma_0^{-1} \mathbf{\mu}_0 + \beta \mathbf{X}^T \mathbf{t}) \\ \Sigma_N^{-1} &= \Sigma_0^{-1} + \beta \mathbf{X}^T \mathbf{X} \end{aligned}$$

where $\mathbf{X}$ is the feature matrix of size $N \times M$.

- What happens with different values of β and Σ_0 ?

To simplify, let's assume the covariance of the prior to be **isotropic**, that is, it is a diagonal matrix with the same value along the diagonal, $\Sigma_0 = \alpha^{-1} \mathbf{I}$. And, let $\mathbf{\mu}_0 = [0, 0]$, thus

$$\mathbf{\mu}_N = \beta \Sigma_N \mathbf{X}^T \mathbf{t}$$

and

$$\Sigma_N^{-1} = \alpha \mathbf{I} + \beta \mathbf{X}^T \mathbf{X}$$

Let's consider the example presented in the Bishop textbook (Figure 3.7 in page 155).

Consider a single input variable $\mathbf{x}$, a single target variable $\mathbf{t}$ and a linear model of the form $y(\mathbf{x}, \mathbf{w}) = w_0 + w_1 \mathbf{x}$. Because this has just two parameters coefficients, $\mathbf{w} = [w_0, w_1]^T$, we can plot the prior and posterior distributions directly in parameter space (2-dimensional parameter space).

Let's generate some synthetic data from the function $f(x, \mathbf{a}) = w_0 + w_1 x$ with parameter values $w_0 = -0.3$ and $w_1 = 0.5$ by first choosing values of x_n from the uniform distribution $U(x_n | -1, 1)$, then evaluating $f(x_n, \mathbf{w})$, and finally adding Gaussian noise with standard deviation of $\sigma = 0.2$ to obtain the target values t_n .

$$t_n = f(x_n, \mathbf{w}) + \epsilon = -0.3 + 0.5 x_n + \epsilon$$

where $\epsilon \sim \mathcal{N}(0, \beta \mathbf{I})$.

- **Our goal is to recover the values of w_0 and w_1 from such data, and we will explore the dependence on the size of the data set.**

For some data, $\{x_n, t_n\}_{n=1}^N$, we can pose this problem in terms of

Regularized Least Squares:

$$\begin{aligned} J(\mathbf{w}) &= \frac{1}{2} \sum_{n=1}^N (t_n - y_n)^2 + \frac{\lambda}{2} \sum_{i=0}^1 w_i^2 \\ &= \frac{1}{N} \sum_{n=1}^N (t_n - y_n)^2 + \frac{\lambda}{2} (w_0^2 + w_1^2) \end{aligned} \quad \rightarrow \arg\min_{\mathbf{w}} J(\mathbf{w})$$

- Using **MAP**, we can rewrite our objective using the **Bayesian interpretation**:

$$\begin{aligned} \arg_{\mathbf{w}} \max P(\mathbf{\epsilon}|\mathbf{w})P(\mathbf{w}) \end{aligned}$$

Let's consider the data likelihood, $P(\mathbf{\epsilon}|\mathbf{w})$, to be a Gaussian distribution with mean $\mu = 0$ and variance $\sigma^2 = \beta$. And let's also consider the prior distribution, $P(\mathbf{w})$, to be a Gaussian distribution with mean μ_0 and variance $\sigma_0^2 = \alpha^{-1}$. Then, using the derivations from above, we can rewrite our optimization as:

$$\arg_{\mathbf{w}} \max \mathcal{N}(\mathbf{\epsilon}|0, \beta) \mathcal{N}(\mathbf{w}|\mu_0, \alpha^{-1}) \propto \arg_{\mathbf{w}} \max \mathcal{N}(\mathbf{w}|\mu_0, \Sigma_N^{-1})$$

where $\mu_0 = [0, 0]$, $\mathbf{X}$ is the polynomial feature matrix, and $\Sigma_N = (\alpha^{-1} \mathbf{I} + \beta \mathbf{X}^T \mathbf{X})^{-1}$ is the covariance matrix of the posterior distribution.

Note that we **do not know** the parameters of the prior distribution (μ_0 and σ_0 are unknown). The parameters of the prior distribution will have to be chosen by the user. And they will essentially *encode* any behavior or a priori knowledge we may have about the weights.

- **Both our data likelihood and prior distributions are in a 2-dimensional space (this is because our *model order* is $M=2$ -- we have 2 parameters!).**

We are going to generate data from $t = -0.3 + 0.5x + \epsilon$ where ϵ is drawn from a zero-mean Gaussian distribution.

- **The goal is to estimate the values $w_0 = -0.3$ and $w_1 = 0.5$**
- The feature matrix $\mathbf{X}$ can be computed using the polynomial basis functions
- **Parameters to choose:** β and α

We want to implement this scenario for a case that we are getting more data every minute. As we get more and more data, we want to **update our prior distribution using our posterior distribution (informative prior)**, because they take the have the same distribution form. This is only possible because Gaussian-Gaussian have a conjugate prior relationship. That is, the posterior distribution is also a Gaussian.

```
In [8]: from scipy.stats import multivariate_normal
import textwrap
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
```

```

def likelihood_prior_func(beta = 2, alpha = 1, prior_mean=[0,0],
                           draw_num=(0,1,10,20,50,100), rnd_seed=9, prior_upd
rng = np.random.default_rng(rnd_seed)
'''Online Update of the Posterior distribution for a Gaussian-Gaussian c
Parameter:
beta - variance of the data likelihood (of the additive noise)
alpha - precision value or 1/variance of the prior distribution
draw_num - number of points collected at each instance.

This function will update the prior distribution as new data points are
The prior distribution will be the posterior distribution from the last

fig = plt.figure(figsize=(20, 20), dpi= 80, facecolor='w', edgecolor='k'

# true (unknown) weights
a = -0.3 # w0
b = 0.5 # w1

# set up input space
rangeX = [-2, 2] # range of values for the input
step = 0.025 # distance between points
X = np.mgrid[rangeX[0]:rangeX[1]:step] # creates a grid of values for in

#initialize prior/posterior and sample data
S0 = (1/alpha)*np.eye(2) # prior covariance matrix
sigma = S0 # copying it so we can update it later
mean = prior_mean # mean for prior

# Draws samples from Uniform(-1,1) distribution
draws = rng.uniform(rangeX[0],rangeX[1],size=draw_num[-1])
# Generate the noisy target samples
T = a + b*draws + rng.normal(loc=0, scale=np.sqrt(beta))

for i in range(len(draw_num)):
    if draw_num[i]>0: #skip first image

        # INPUT DATA
        #Feature Matrix (Polynomial features with M=2)
        FeatureMatrix = np.array([draws[:draw_num[i]]**m for m in range(
        #Target Values
        t = T[0:draw_num[i]]

        # POSTERIOR PROBABILITY
        # Covariance matrix
        sigma = np.linalg.inv(np.linalg.inv(S0) + beta*FeatureMatrix.T@F
        # Mean vector
        mean = beta*sigma@FeatureMatrix.T@t

        # PARAMETER SPACE
        # create a meshgrid of possible values for w's
        w0, w1 = np.mgrid[rangeX[0]:rangeX[1]:step, rangeX[0]:rangeX[1]:

        # Define the Gaussian distribution for data likelihood
        p = multivariate_normal(mean=t[draw_num[i]-1], cov=beta)
        # Initialize the PDF for data likelihood
        out = np.empty(w0.shape)

```

```

# For each value (w0,w1), compute the PDF for all data samples
for j in range(len(w0)):
    out[j] = p.pdf(w0[j]+w1[j]*draws[draw_num[i]-1])

# Plot the data likelihood
ax = fig.add_subplot(*[len(draw_num),3,(i)*3+1])
ax.pcolor(w0, w1, out)
# Add the current value for parameters w=(w0,w1)
ax.scatter(a,b, c='r',marker='x')
myTitle = 'data likelihood'
ax.set_title("\n".join(textwrap.wrap(myTitle, 100)))

if prior_update:
    # Prior update
    S0 = sigma

# PARAMETER SPACE
# create a meshgrid of possible values for w's
w0, w1 = np.mgrid[rangeX[0]:rangeX[1]:step, rangeX[0]:rangeX[1]:step

# POSTERIOR PROBABILITY
# initialize the matrix with posterior PDF values
pos = np.empty(w1.shape + (2,))
# for w0
pos[:, :, 0] = w0
# and for w1
pos[:, :, 1] = w1
# compute the PDF
p = multivariate_normal(mean=mean, cov=sigma)

#Show prior/posterior
ax = fig.add_subplot(*[len(draw_num),3,(i)*3+2])
ax.pcolor(w0, w1, p.pdf(pos))
# Add the value for parameters w=(w0,w1) that MAXIMIZE THE POSTERIOR
ax.scatter(a,b, c='r',marker='x')
myTitle = 'Prior/Posterior'
ax.set_title("\n".join(textwrap.wrap(myTitle, 100)))

# DATA SPACE
ax = fig.add_subplot(*[len(draw_num),3,(i)*3+3])
for j in range(6):
    # draw sample from the prior probability to generate possible values
    w0, w1 = np.random.multivariate_normal(mean=mean, cov=sigma)
    # Estimated labels
    t = w0 + w1*X
    # Show data space
    ax.plot(X,t)
    if draw_num[i] > 0:
        ax.scatter(FeatureMatrix[:,1], T[0:draw_num[i]])
    myTitle = 'data space'
    ax.set_title("\n".join(textwrap.wrap(myTitle, 100)))

```

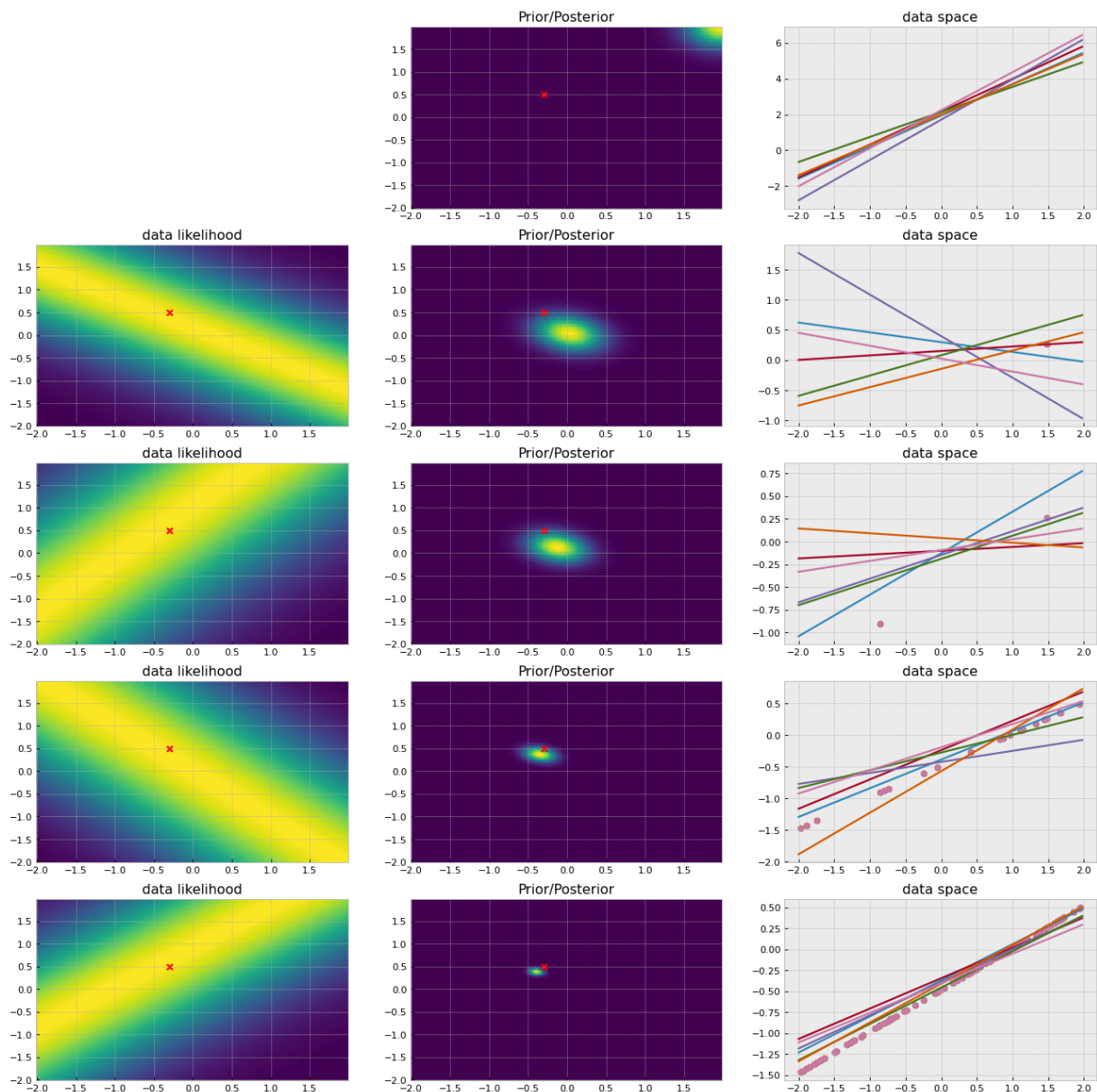
In [9]:

```

# Parameters
# beta - variance of the data likelihood (for the additive noise)
# alpha - precision value or 1/variance for the prior distribution
# draw_num - number of points collected at each instance

```

```
likelihood_prior_func(beta = 2, alpha = 1/0.1, prior_mean= [2,2],
                     draw_num=(0,1,2,20,100), prior_update=1)
```



Introduction to Classification & Probabilistic Generative Models

So far we have focused on regression. We will begin to discuss **classification**.

Suppose we have training data from two classes, C_1 and C_2 , and we would like to train a classifier to assign a label to incoming test points as C_1 or C_2 .

There are *many* classifiers in the machine learning literature. We will cover a few in this course. Today we will focus on probabilistic generative approaches for classification.

There are two types of classification algorithms: **discriminative** or **generative**.

- A **discriminative** approach for classification is one in which we partition the feature space into regions for each class. Then, when we have a test point, we evaluate in which region it landed on and classify it accordingly.
- A **generative** approach for classification is one in which we estimate the parameters for distributions that generate the data for each class. Then, when we have a test point, we can compute the posterior probability of that point belonging to each class and assign the point to the class with the highest posterior probability.

```
In [31]: def generateData(mean1, mean2, cov1, cov2, N1, N2):
# We are generating data from two Gaussians to represent two classes
# In practice, we would not do this - we would just have data from the p
data_C1 = stats.multivariate_normal(mean1, cov1).rvs(size=N1)
data_C2 = stats.multivariate_normal(mean2, cov2).rvs(size=N2)

# Entire Training Dataset
data = np.concatenate((data_C1, data_C2))
labels = np.concatenate((np.ones(N1), 2*np.ones(N2)))

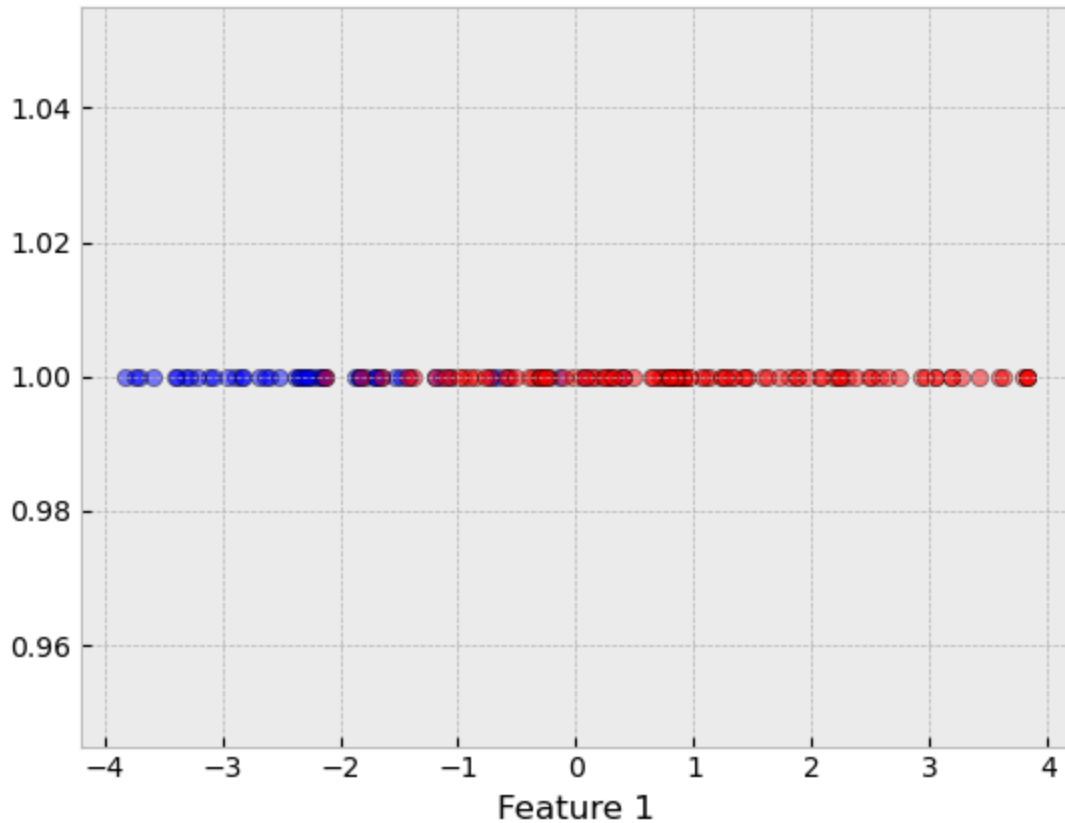
if len(mean1)>1:
    plt.scatter(data[labels==1,0], data[labels==1,1], c='b', alpha=0.5,
    plt.scatter(data[labels==2,0], data[labels==2,1], c='r', alpha=0.5,
    plt.xlabel('Feature 1'); plt.ylabel('Feature 2');
else:
    plt.scatter(data[labels==1], np.ones(N1), c='b', alpha=0.5, edgecolor
    plt.scatter(data[labels==2], np.ones(N2), c='r', alpha=0.5, edgecolor
    plt.xlabel('Feature 1');

return data, labels
```

Case 1: Univariate Data Likelihood

```
In [34]: mean1 = [-2]
mean2 = [1]
var1 = [1]
var2 = [2]
N1 = 50 # C1 - blue
N2 = 100 # C2 - red

data, labels = generateData(mean1, mean2, var1, var2, N1, N2)
```



```
In [37]: def drawMAP(data, labels):

    ### Under the assumption that red follows a Gaussian distribution, and blue follows a Gaussian distribution
    #### Estimate parameters (MLE solution)
    # Means
    mu1 = np.mean(data[labels==1])
    mu2 = np.mean(data[labels==2])

    # Variances
    var1 = np.cov(data[labels==1])
    var2 = np.cov(data[labels==2])

    #### Estimate Prior Probabilities - relative frequency
    N = len(data)
    N1 = np.sum(labels==1)
    N2 = N - N1
    p1 = 0.8 # prior probability for C1
    p2 = 0.2 # prior probability for C2

    #### Define data likelihoods
    G1=stats.norm(loc=mu1,scale=np.sqrt(var1)) # P(x|C1)
    G2=stats.norm(loc=mu2,scale=np.sqrt(var2)) # P(x|C2)
    x=np.linspace(-6,6,1001)

    #### Plot the weighted densities
    # these are proportional to the posteriors
    plt.figure(figsize=(10,5))
    plt.plot(x,p1*G1.pdf(x),label='$f_X(x|C_1)P(C_1)$')
    plt.plot(x,p2*G2.pdf(x),label='$f_X(x|C_2)P(C_2)$')
```

```

#### Determine the regions where the posterior for deciding C1
# and the posterior for deciding C2
R1=x[np.where(p1*G1.pdf(x)>= p2*G2.pdf(x))]
R2=x[np.where(p1*G1.pdf(x)< p2*G2.pdf(x))]

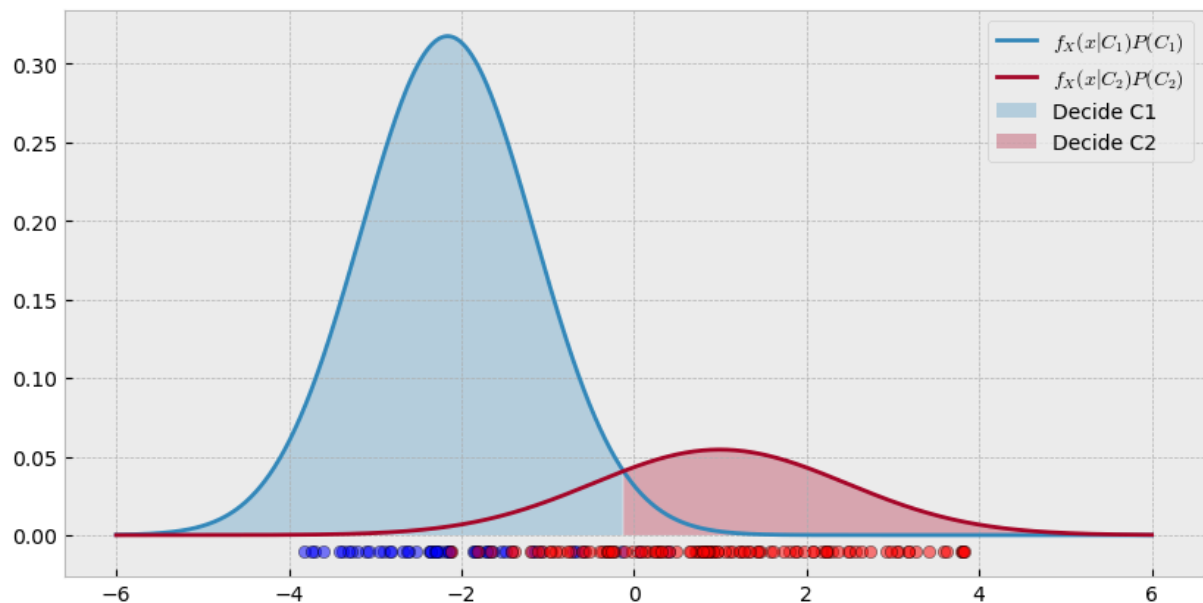
# Fill under the regions found above
plt.fill_between(R1,p1*G1.pdf(R1),alpha=0.3,label='Decide C1')
plt.fill_between(R2,p2*G2.pdf(R2),alpha=0.3,label='Decide C2')
plt.scatter(data[labels==1], -0.01*np.ones(N1), c='b', alpha=0.5, edgecc
plt.scatter(data[labels==2], -0.01*np.ones(N2), c='r', alpha=0.5, edgecc
plt.legend()

# Print the MAP threshold
print('MAP decision threshold to decide C2 is >',round(R2[0],2))

```

In [38]: drawMAP(data,labels)

MAP decision threshold to decide C2 is > -0.12



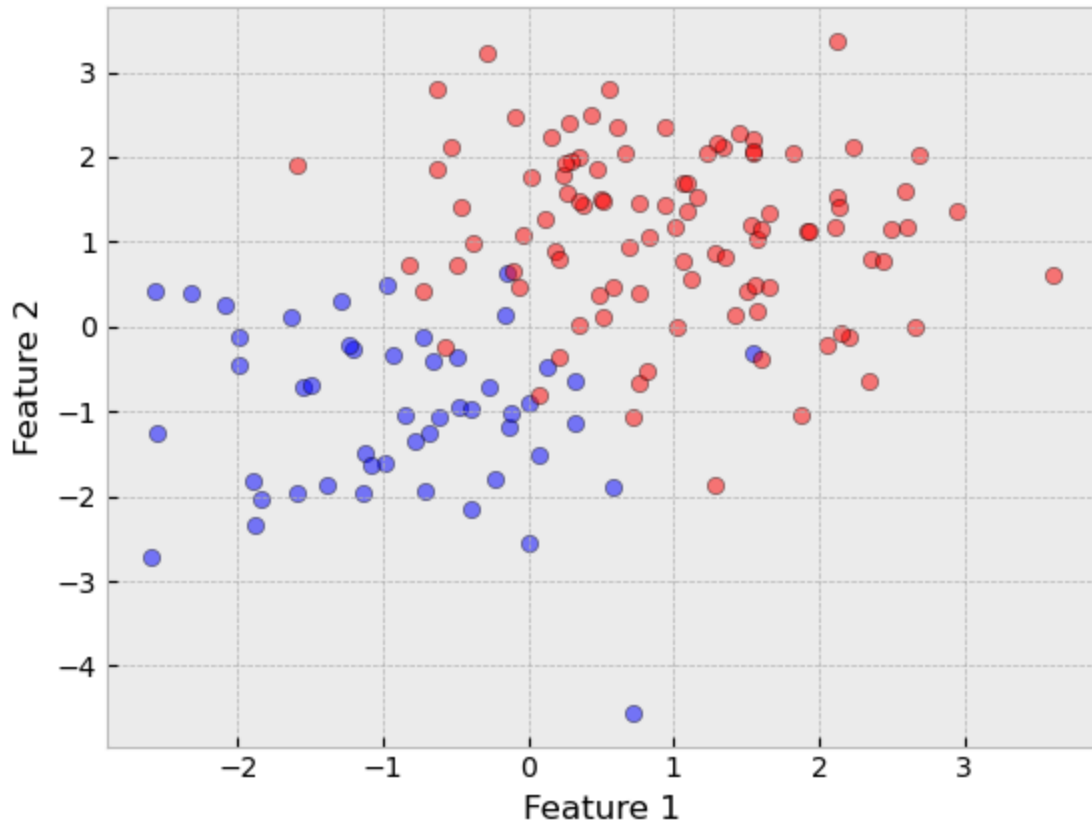
Case 2: Multivariate Data Likelihood

```

In [39]: mean1 = [-1, -1]
mean2 = [1, 1]
cov1 = [[1,0],[0,1]]
cov2 = [[1,0],[0,1]]
N1 = 50
N2 = 100

data, labels = generateData(mean1, mean2, cov1, cov2, N1, N2)

```



In the data we generated above, we have a "red" class and a "blue" class. When we are given a test sample, we will want to assign the label of red or blue.

We can compute the **posterior probability** for class C_1 as follows:

$$P(C_1|x) = \frac{P(x|C_1)P(C_1)}{P(x)}$$

Understanding that the two classes, red and blue, form a partition of all possible classes, then we can utilize the *Law of Total Probability*, and obtain:

$$P(C_1|x) = \frac{P(x|C_1)P(C_1)}{P(x|C_1)P(C_1) + P(x|C_2)P(C_2)}$$

Similarly, we can compute the posterior probability for class C_2 :

$$P(C_2|x) = \frac{P(x|C_2)P(C_2)}{P(x|C_1)P(C_1) + P(x|C_2)P(C_2)}$$

Note that $P(C_1|x) + P(C_2|x) = 1$.

The Naive Bayes Classifier

Therefore, for a given test point $\mathbf{x}^*$, our decision rule is:

$$P(C_1|\mathbf{x}^*) \underset{C_2}{\overset{C_1}{>}} P(C_2|\mathbf{x}^*)$$

Using the Bayes' rule, we can further rewrite it as:
$$\frac{P(\mathbf{x}^*|C_1)P(C_1)}{P(\mathbf{x}^*)} \gtrless \frac{P(\mathbf{x}^*|C_2)P(C_2)}{P(\mathbf{x}^*)} \iff P(\mathbf{x}^*|C_1)P(C_1) \gtrless P(\mathbf{x}^*|C_2)P(C_2)$$

We assign $\mathbf{x}^*$ as class 1 if $p(\mathbf{x}^*|C_1)p(C_1) > p(\mathbf{x}^*|C_2)p(C_2)$, or assign $\mathbf{x}^*$ to class 2 if $p(\mathbf{x}^*|C_1)p(C_1) < p(\mathbf{x}^*|C_2)p(C_2)$.

This defines the **Naive Bayes Classifier**.

Training a Generative Classifier

- So, **to train the classifier**, what we need to do is to determine the parametric forms and the associated parameters for $P(\mathbf{x}|C_1)$, $P(\mathbf{x}|C_2)$, $P(C_1)$ and $P(C_2)$.

For example, we can assume that the data samples coming from either C_1 and C_2 are distributed according to Gaussian distributions. In this case,

$$P(\mathbf{x}|C_k) = \frac{1}{(2\pi)^{1/2} |\Sigma_k|^{1/2}} \exp\left\{-\frac{1}{2}(\mathbf{x} - \mu_k)^T \Sigma_k^{-1} (\mathbf{x} - \mu_k)\right\}, \text{ for all } k=\{1,2\}$$

- We can consider any distributional form we want.

What about the $P(C_1)$ and $P(C_2)$?

- We can consider the relative frequency of each class, that is, $P(C_i) = \frac{N_i}{N}$, where N_i is the number of points in class C_i and N is the total number of samples.

MLE Parameter Estimation Steps

For simplification, let's consider the covariance matrix Σ_k for $k=1,2$ to be **isotropic** matrices, that is, the covariance matrix is diagonal and the element along the diagonal is the same, or: $\Sigma_k = \sigma_k^2 \mathbf{I}$.

- What are the parameters? The mean and covariance of the Gaussian distribution for both classes.

Given the assumption of the Gaussian form, how would you estimate the parameters for $P(\mathbf{x}|C_1)$ and $P(\mathbf{x}|C_2)$? We can use **maximum likelihood estimate** for the mean

and covariance, because we are looking for the parameters of the distributions that *maximize* the data likelihood!

Assumption: Assuming the classes follow a (bivariate or 2-D) Gaussian distribution and, for simplicity, let's assume the covariance matrices are **isotropic**, that is, $\Sigma_k = \sigma_k^2 \mathbf{I}$.

The MLE steps for parameter estimation are:

1. Write down the observed data likelihood, $\mathcal{L}^0$

$$\begin{aligned} \mathcal{L}^0 &= P(x_1, x_2, \dots, x_N | C_k) \\ &= \prod_{n=1}^N P(x_n | C_k) \quad \text{data samples are i.i.d.} \\ &= \prod_{n=1}^N \frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}} \exp\left\{-\frac{1}{2} (x_n - \mu_k)^T \Sigma_k^{-1} (x_n - \mu_k)\right\} \\ &= \prod_{n=1}^N \frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}} \exp\left\{-\frac{1}{2\sigma_k^2} (x_n - \mu_k)^T (x_n - \mu_k)\right\} \\ &= \prod_{n=1}^N \frac{1}{(2\pi)^{d/2} (\sigma_k^2)^{d/2}} \exp\left\{-\frac{1}{2\sigma_k^2} (x_n - \mu_k)^T (x_n - \mu_k)\right\} \end{aligned}$$

where d is the dimensionality of the data space.

2. Take the log-likelihood, $\mathcal{L}$. This *trick* helps in taking derivatives.

$$\begin{aligned} \mathcal{L} &= \ln(\mathcal{L}^0) \\ &= \sum_{n=1}^N \left(-\frac{d}{2} \ln 2\pi - \frac{d}{2} \ln \sigma_k^2 - \frac{1}{2\sigma_k^2} (x_n - \mu_k)^T (x_n - \mu_k) \right) \end{aligned}$$

3. Take the derivative of the log-likelihood function with respect to the parameters of interest. For Gaussian distribution they are the mean and covariance.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mu_k} &= 0 \quad \sum_{n \in C_k} \frac{1}{\sigma_k^2} (x_n - \mu_k) = 0 \quad \sum_{n \in C_k} (x_n - \mu_k) = 0 \\ \sum_{n \in C_k} x_n - N_k \mu_k &= 0 \quad \sum_{n \in C_k} x_n = N_k \mu_k \end{aligned}$$

This is the sample mean for each class. And,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \sigma_k^2} &= 0 \quad \sum_{n \in C_k} -\frac{d}{2\sigma_k^2} + \frac{(x_n - \mu_k)^T (x_n - \mu_k)}{(\sigma_k^2)^2} = 0 \\ \sum_{n \in C_k} -\frac{d}{2} + \frac{(x_n - \mu_k)^T (x_n - \mu_k)}{\sigma_k^2} &= 0 \\ \sum_{n \in C_k} (x_n - \mu_k)^T (x_n - \mu_k) &= d N_k \sigma_k^2 \\ \sigma_k^2 &= \frac{\sum_{n \in C_k} (x_n - \mu_k)^T (x_n - \mu_k)}{d N_k} \end{aligned}$$

This is the sample variance for each class. Then we can create $\Sigma_k = \sigma_k^2 \mathbf{I}$, which is the (biased) sample covariance for each class.

In practice, if we want to estimate an entire covariance matrix, we would have to take the derivative of the log-likelihood function with respect to every entry in the covariance

matrix. Covariance matrices are symmetric, so we only need to determine the upper (or lower) half of the covariance matrix.

We can determine the values for $p(C_1)$ and $p(C_2)$ from the number of data points in each class:

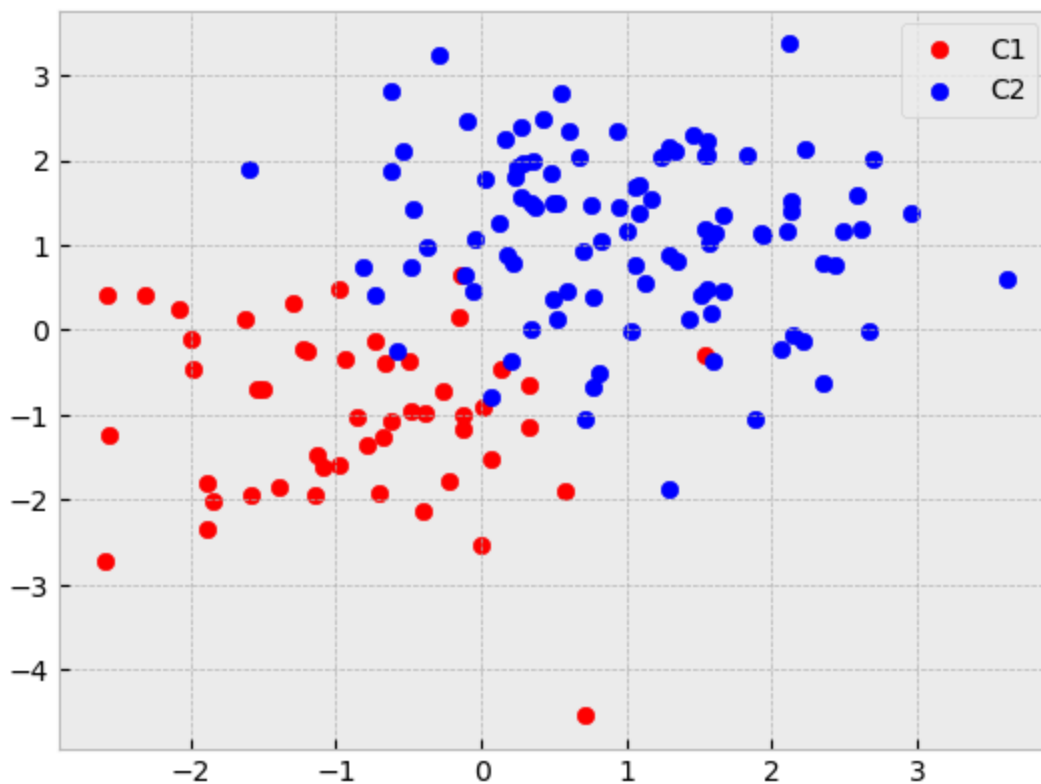
$$p(C_k) = \frac{N_k}{N}$$

where N is the total number of data points.

```
In [40]: data.shape
```

```
Out[40]: (150, 2)
```

```
In [41]: plt.scatter(data[labels==1,0], data[labels==1,1], color='red', label='C1')
plt.scatter(data[labels==2,0], data[labels==2,1], color='blue', label='C2')
plt.legend();
```



to be continued...
